

Mise en place de l'authentification

Jegonflable Aube — Guide stagiaire

Ce guide t'apprend à mettre en place une authentification complète pour le back-office admin du site, étape par étape, en partant de l'existant. À la fin, tu pourras te connecter avec `max@webomax.fr`, accéder à `/admin` et te déconnecter.

Ce qui est déjà en place :

- `lib/auth.ts` — config NextAuth (CredentialsProvider + JWT)
- `app/api/auth/[...nextauth]/route.ts` — handler API
- `types/next-auth.d.ts` — types étendus pour la session
- `lib/validators/auth.ts` — schéma Zod pour le login
- `prisma/seed.ts` — admin créé en base

Ce que tu vas créer dans ce guide :

- Un **SessionProvider** pour rendre la session accessible dans toute l'app
- Une page `/login` avec formulaire et gestion d'erreurs
- Un **middleware** qui protège `/admin/*`
- Un **layout admin** avec barre d'utilisateur et bouton déconnexion
- Une page `/admin` de test pour valider que tout fonctionne

■ **Note** — Avant de commencer : ton seed doit être lancé (`npm run db:seed`) pour que l'admin existe en base. Et ton serveur doit tourner (`npm run dev`).

Plan

- **1.** Comprendre ce qu'on va construire
- **2.** Étape 1 — Créer le SessionProvider
- **3.** Étape 2 — Brancher le SessionProvider dans le layout racine
- **4.** Étape 3 — Créer la page /login
- **5.** Étape 4 — Créer le middleware de protection
- **6.** Étape 5 — Créer le layout /admin
- **7.** Étape 6 — Créer la page /admin de test
- **8.** Étape 7 — Tester le flux complet
- **9.** Le bug planqué (BUG-04) — pourquoi tu peux pas te connecter au début
- **10.** Annexe — comment ça marche sous le capot

1. Comprendre ce qu'on va construire

L'authentification de ce projet utilise **NextAuth v4** avec la stratégie **Credentials** (email + mot de passe en DB) et des sessions stockées dans un **JWT** (cookie signé, pas de table session en base).

Schéma du flux

1. Utilisateur ouvre /admin
2. Le middleware vérifie le JWT dans les cookies
 - pas de JWT -> redirect /login
 - JWT valide -> laisse passer
3. Sur /login, l'utilisateur soumet email + password
4. Côté client : `signIn('credentials', { email, password })`
5. NextAuth appelle `authorize()` dans `lib/auth.ts`
6. `authorize()` lit le User en DB, compare le hash bcrypt
7. Si OK -> JWT créé + cookie posé
8. Redirect vers /admin

Les 3 zones de l'app

Zone	Route	Protégée ?
Public	/, /jeux, /devis	Non
Auth	/login	Non (mais redirige si déjà connecté)
Admin	/admin/*	Oui — middleware

Étape 1 — Créer le SessionProvider

Pourquoi ? Le hook `useSession()` a besoin d'un `<SessionProvider>` au-dessus dans l'arbre React. Mais comme c'est un composant client et que `app/layout.tsx` doit rester serveur (pour le SEO, les metadata), on l'isole dans un fichier client à part.

■ `components/providers/SessionProvider.tsx`

```
"use client";

import { SessionProvider as NextAuthSessionProvider } from "next-auth/react";

export function SessionProvider({ children }: { children: React.ReactNode }) {
  return <NextAuthSessionProvider>{children}</NextAuthSessionProvider>;
}
```

Ce que ça fait : on importe le `SessionProvider` de `NextAuth` et on le ré-exporte dans un composant marqué `"use client"`. Ça permettra à `app/layout.tsx` (serveur) de l'utiliser sans devenir client lui-même.

Étape 2 — Brancher le SessionProvider

Ouvre `app/layout.tsx`. Tu dois envelopper `{children}` dans le `SessionProvider`.

■ `app/layout.tsx (modification)`

```
import { SessionProvider } from "@/components/providers/SessionProvider";

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="fr">
      <body>
        <SessionProvider>
          {children}
        </SessionProvider>
      </body>
    </html>
  );
}
```

■ **Note** — Le reste du layout (fonts, metadata, classes Tailwind) reste tel quel. Tu n'ajoutes que l'import et le wrapping.

Étape 3 — Créer la page /login

Page client avec un formulaire email/mot de passe. On utilise `signIn()` de NextAuth qui gère tout : l'appel à la route API, la création du cookie JWT, et la redirection.

■ `app/login/page.tsx`

```
"use client";

import { useState } from "react";
import { useRouter, useSearchParams } from "next/navigation";
import { signIn } from "next-auth/react";
import { Button } from "@/components/ui/Button";
import { Input } from "@/components/ui/Input";

export default function LoginPage() {
  const router = useRouter();
  const searchParams = useSearchParams();
  const callbackUrl = searchParams.get("callbackUrl") ?? "/admin";

  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState<string | null>(null);
  const [loading, setLoading] = useState(false);

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    setError(null);
    setLoading(true);

    const res = await signIn("credentials", {
      email,
      password,
      redirect: false,
    });

    setLoading(false);

    if (res?.error) {
      setError("Email ou mot de passe invalide");
      return;
    }

    router.push(callbackUrl);
    router.refresh();
  }

  return (
    <main className="flex min-h-screen items-center justify-center p-4">
      <form onSubmit={handleSubmit} className="w-full max-w-sm space-y-4 rounded-2xl bg-white p-8 shadow-lg">
        <h1 className="font-heading text-2xl font-bold text-primary-900">Connexion</h1>

        <Input
          type="email"
          placeholder="Email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          required
        />
        <Input
          type="password"
          placeholder="Mot de passe"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
        />
      </form>
    </main>
  );
}
```

```
required
/>

{error && <p className="text-sm text-error">{error}</p>}

<Button type="submit" fullWidth disabled={loading}>
{loading ? "Connexion..." : "Se connecter"}
</Button>
</form>
</main>
);
}
```

Points d'attention

- `redirect: false` — sinon NextAuth fait un hard reload, on perd le contrôle des erreurs
- `callbackUrl` — quand le middleware redirige vers `/login`, il passe l'URL d'origine en query param, on s'en sert pour rediriger après login
- `router.refresh()` — force la revalidation des Server Components avec la nouvelle session

Étape 4 — Créer le middleware de protection

Le middleware tourne **avant** chaque request, côté serveur Edge. Idéal pour vérifier le JWT et rediriger si nécessaire. NextAuth fournit un helper qui fait tout le boulot : `withAuth`.

■ `middleware.ts` (à la racine du projet, pas dans `app/`)

```
import { withAuth } from "next-auth/middleware";

export default withAuth({
  pages: {
    signIn: "/login",
  },
});

// Quelles routes le middleware doit protéger
export const config = {
  matcher: ["/admin/:path*"],
};
```

Comment ça marche

- À chaque request sur `/admin/*`, NextAuth lit le cookie JWT
- Si pas de JWT ou JWT invalide → redirect `/login?callbackUrl=...`
- Si JWT valide → la request passe, le composant se charge

■ **Attention** — Le fichier doit s'appeler exactement `middleware.ts` et être à la racine du projet (au même niveau que `package.json`), pas dans `app/`. Sinon Next.js ne le détecte pas.

Étape 5 — Créer le layout `/admin`

Un layout pour toutes les pages `/admin/*` avec une barre d'utilisateur et un bouton déconnexion. On utilise `getSession()` côté serveur pour récupérer la session (et faire une 2ème vérif en plus du middleware — défense en profondeur).

■ `app/admin/layout.tsx`

```
import { getSession } from "next-auth";
import { redirect } from "next/navigation";
import { authOptions } from "@/lib/auth";
import { LogoutButton } from "@/components/admin/LogoutButton";

export default async function AdminLayout({ children }: { children: React.ReactNode }) {
  const session = await getSession(authOptions);

  if (!session) {
    redirect("/login");
  }

  return (
    <div className="min-h-screen bg-gray-50">
      <header className="border-b bg-white px-6 py-3">
        <div className="mx-auto flex max-w-6xl items-center justify-between">
          <h1 className="font-heading text-lg font-bold text-primary-900">
            Back-office
          </h1>
          <div className="flex items-center gap-4 text-sm">
            <span className="text-gray-600">
              {session.user.name ?? session.user.email}
            </span>
            <LogoutButton />
          </div>
        </div>
      </header>
      <div>{children}</div>
    </div>
  );
}
```

```
</header>
<main className="mx-auto max-w-6xl p-6">{children}</main>
</div>
);
}
```


Et le bouton déconnexion (composant client séparé) :

■ components/admin/LogoutButton.tsx

```
"use client";

import { signOut } from "next-auth/react";
import { Button } from "@components/ui/Button";

export function LogoutButton() {
  return (
    <Button
      variant="ghost"
      size="sm"
      onClick={() => signOut({ callbackUrl: "/" })}
    >
      Déconnexion
    </Button>
  );
}
```

Étape 6 — Créer la page /admin (test)

Page de bienvenue minimaliste pour vérifier que tout marche.

■ app/admin/page.tsx

```
import { getServerSession } from "next-auth";
import { authOptions } from "@lib/auth";

export default async function AdminHomePage() {
  const session = await getServerSession(authOptions);

  return (
    <div className="rounded-2xl bg-white p-8 shadow">
      <h2 className="mb-2 font-heading text-2xl font-bold text-primary-900">
        Bienvenue {session?.user.name}
      </h2>
      <p className="text-gray-600">
        Tu es connecté en tant que <b>{session?.user.role}</b>.
      </p>
    </div>
  );
}
```

Étape 7 — Tester le flux complet

Lance l'app si ce n'est pas déjà fait :

```
npm run dev
```

Test 1 — Accès admin sans connexion

- Ouvre `http://localhost:3000/admin`
- Tu dois être redirigé vers `/login?callbackUrl=%2Fadmin`
- ✓ Le middleware fonctionne

Test 2 — Login avec mauvais identifiants

- Tape `n'importe@quoi.fr + blabla`
- Tu dois voir « *Email ou mot de passe invalide* »
- ✓ La gestion d'erreur fonctionne

Test 3 — Login OK

- Tape max@webomax.fr + WebomaxAdmin2026!
- Tu dois être redirigé vers /admin
- Tu dois voir « *Bienvenue Maxime — WEBOMAX* »
- ✓ L'auth fonctionne de bout en bout

Test 4 — Déconnexion

- Clique sur « Déconnexion »
- Tu dois être redirigé vers /
- Si tu retournes sur /admin → re-redirect vers /login
- ✓ La déconnexion fonctionne

9. Le bug planqué — pourquoi tu peux pas te connecter

■ **Attention** — Si tu suis ce guide à la lettre et que le Test 3 échoue avec « *Email ou mot de passe invalide* » alors que tes identifiants sont corrects, ce n'est pas ta faute. C'est un bug pédagogique volontaire.

Symptôme

Tu connais bien les identifiants (ils sont affichés à la fin du seed), mais le login refuse systématiquement.

Indices progressifs

Indice 1 — Cherche les commentaires BUG- dans le projet :

```
grep -rn "BUG-" --include="*.ts" --include="*.tsx" .
```

Tu vas en trouver plusieurs. Deux concernent l'auth : BUG-02 (dans `seed.ts`) et BUG-04 (dans `lib/auth.ts`).

Indice 2 — Lis attentivement la fonction `authorize()` dans `lib/auth.ts` :

```
// BUG-04 : comparaison de mot de passe en clair au lieu d'utiliser bcrypt.compare
// Conséquence : impossible de se connecter (le hash bcrypt en DB
// ne sera jamais égal au mot de passe en clair saisi).
const isValid = credentials.password === user.password;
```

Indice 3 — la solution

Il y a en fait **deux bugs liés** qui s'annulent dans certains cas :

- **BUG-02** dans `prisma/seed.ts` ligne 28 — le mot de passe admin est stocké en clair (`password: adminPassword`) au lieu d'être hashé
- **BUG-04** dans `lib/auth.ts` ligne 40 — la comparaison utilise `===` au lieu de `bcrypt.compare()`

Le fix correct (les deux ensemble) :

Dans `prisma/seed.ts` :

```
// AVANT
const adminPassword = "WebomaxAdmin2026!";
// ...
create: { ..., password: adminPassword, ... }

// APRÈS
const adminPassword = await bcrypt.hash("WebomaxAdmin2026!", 10);
// ...
create: { ..., password: adminPassword, ... }
```

Dans `lib/auth.ts` :

```
// AVANT
const isValid = credentials.password === user.password;

// APRÈS
const isValid = await bcrypt.compare(credentials.password, user.password);
```

Puis relance le seed pour régénérer l'admin avec le hash :

```
npx prisma db push --force-reset # vide la DB (attention en prod !)
npm run db:seed # recrée tout proprement
```

✓ Avec ces deux fixes, le login fonctionne. C'est aussi pour ça que bcryptjs est déjà dans les dépendances : il était prévu pour être utilisé.

Annexe — Comment ça marche sous le capot

Le cookie de session

Après login, NextAuth pose un cookie nommé `next-auth.session-token` (ou `__Secure-next-auth.session-token` en HTTPS) qui contient un JWT signé avec `NEXTAUTH_SECRET`.

Tu peux le voir dans Chrome DevTools → onglet Application → Cookies.

Côté Server Component

Pour lire la session dans une page serveur, on utilise `getSession(authOptions)`. C'est une fonction async qui décode le JWT et retourne l'objet session typé.

Côté Client Component

Pour lire la session dans un composant client, on utilise le hook `useSession()` :

```
"use client";
import { useSession } from "next-auth/react";

export function UserBadge() {
  const { data: session, status } = useSession();
  if (status === "loading") return <span>...</span>;
  if (!session) return null;
  return <span>{session.user.email}</span>;
}
```

Pourquoi JWT et pas une session DB ?

Plus simple : pas de table session à gérer, pas de cleanup. Convient bien pour ce projet (peu d'utilisateurs, pas besoin de révocation immédiate côté serveur). Pour un projet plus gros avec besoin de logout instantané partout, on basculerait sur session: `{ strategy: "database" }` + `@auth/prisma-adapter` (déjà installé d'ailleurs).

Récap des fichiers à créer

Fichier	Type	Lignes
<code>components/providers/SessionProvider.tsx</code>	Client	~6
<code>app/login/page.tsx</code>	Client	~70
<code>middleware.ts</code>	Edge	~10
<code>app/admin/layout.tsx</code>	Server	~30
<code>components/admin/LogoutButton.tsx</code>	Client	~13
<code>app/admin/page.tsx</code>	Server	~15

Le commit recommandé

```
git add components/providers app/login middleware.ts \  
app/admin components/admin  
git commit -m "feat(auth): mise en place login + middleware + layout admin"
```

Et après ?

Une fois l'auth en place, tu peux attaquer :

- Le dashboard admin (liste des devis, des jeux)
- Les pages CRUD (créer/modifier/supprimer un jeu)
- Le contrôle d'accès par rôle (SUPER_ADMIN vs ADMIN) via une vérif `session.user.role` dans tes Server Actions

Document généré pour le stagiaire — Maxime / WEBOMAX. Mise en place complète de l'authentification NextAuth.